Instead of using unit testing, I try to keep my minimal API endpoints, controllers, and Razor Pages as thin as possible. I push as much of the behavior in these classes into business logic services that can be easily unit-tested, or into middleware and filters, which can be more easily tested independently.

NOTE This is a personal preference. Some people like to get as close to 100 percent test coverage for their code base as possible, but I find testing orchestration classes is often more hassle than it's worth.

Although I tend to forgo unit-testing my ASP.NET Core endpoints, I often write integration tests that test them in the context of a complete application. In the next section, we'll look at ways to write integration tests for your app so you can test its various components in the context of the ASP.NET Core framework as a whole.

36.3 Integration testing: Testing your whole app in-memory

In this section you'll learn how to create integration tests that test component interactions. You'll learn to create a `TestServer` that sends HTTP requests in-memory to test custom middleware components more easily. You'll then learn how to run integration tests for a real application, using your real app's configuration, services, and middleware pipeline. Finally, you'll learn how to use `WebApplicationFactory` to replace services in your app with test versions to avoid depending on third-party APIs in your tests.

If you search the internet for types of testing, you'll find a host of types to choose among. The differences are sometimes subtle, and people don't